
SOEN 6011 - Deliverable 2

F1: $\arccos(x)$ Calculator

Mohammad Aliyaware Khan

Student ID: 40309082 | Concordia University, Montreal

Summer 2026

Deliverable 2 Scope

Problem 5

Implementation

- Tkinter GUI
- From-scratch $\arccos(x)$
- Validation and recovery

Problem 6

Repository

- Public GitHub
- README
- Meaningful commits

Problem 7

Requirements

- Updated D2 requirements
- Testable statements
- Code evidence

Key D2 Outcome

The D1 textual calculator became a Tkinter GUI with range reduction, numerical verification, error recovery, and repository documentation.

Retained D1 Constraints

Valid domain: $[-1, 1]$ Principal range: $[0, \pi]$ Accuracy target: $\leq 10^{-6}$ radians [1]

Key Updated Requirements

ID	Requirement Summary
FR-001–003	Tkinter GUI, numeric input field, and Calculate control.
FR-004–005	Reject non-finite values and values outside $[-1, 1]$.
FR-006–008	Compute $\arccos(x)$ from scratch and display radians in $[0, \pi]$.
FR-009–013	Helpful error messages, recovery, Clear, and Exit controls.
NFR-001	Execute using Python 3.x in an environment that supports Tkinter.
NFR-002	Tested valid results must have absolute error no greater than 10^{-6} radians.
NFR-003–004	Numerical iteration limits and modular separation of responsibilities.

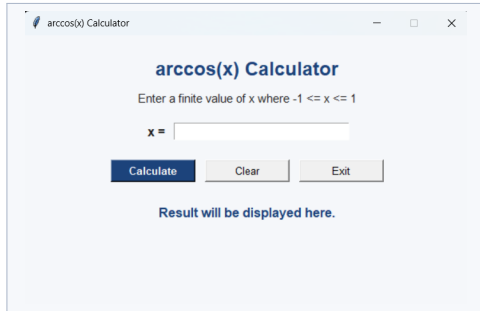
Requirements Evidence

Complete requirements and traceability are presented in the following slides. [\[1, 2\]](#)

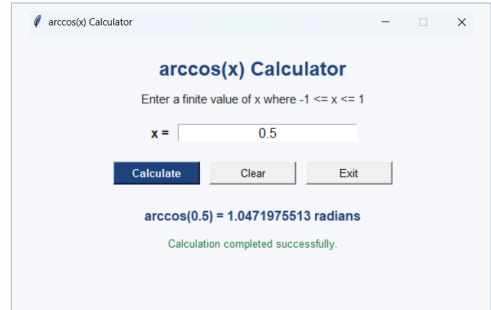
Application Experience

GUI workflow, validation, and error recovery

Tkinter Interface and User Flow



Input interface



Successful calculation

User Flow

Enter x , select **Calculate** or press Enter, receive a result or helpful error message, then correct input and calculate again without restarting.

Input Validation and Error Recovery

Condition	Application Behaviour
Empty input	Requests a finite input in the interval $[-1, 1]$.
Text such as hello	Requests numeric input such as 0, 0.5, or -0.75 .
nan, inf, or -inf	Rejects the value as non-finite.
$x < -1$ or $x > 1$	Explains the valid domain $[-1, 1]$.
Iteration limit reached	Displays a convergence-error message.

Custom Exceptions

InputValidationError
ConvergenceError

Recovery

The GUI catches errors and immediately allows another calculation.

Numerical Design

Range reduction, Taylor recurrence, and Newton's method

Taylor Series with Range Reduction

Why reduction is needed

Direct Taylor-series evaluation of $\arcsin(x)$ converges slowly when $|x|$ is close to 1.

For nonnegative input

$$\arccos(x) = 2 \arcsin \left(\sqrt{\frac{1-x}{2}} \right)$$

The transformed input becomes small when x is close to 1.

[3, 4]

For negative input

$$\arccos(x) = \pi - \arccos(-x)$$

The negative branch is converted to a nonnegative calculation.

From-scratch components

- $\arcsin(x)$: Taylor recurrence
- $\sqrt{x}$: Newton's method
- Endpoints: direct returns

Final Code: Range-Reduced arccos

```
def calculate_arccos(x):
    if not is_finite_number(x):
        raise InputValidationError(
            "Enter a finite number. Values such as nan are not allowed."
        )

    if x < -1.0 or x > 1.0:
        raise InputValidationError(
            "Input is outside the valid domain. Enter a value from -1 to 1."
        )

    if x == 1.0:
        return 0.0
    if x == -1.0:
        return PI
    if x < 0.0:
        return PI - calculate_arccos(-x)

    reduced_input = calculate_square_root((1.0 - x) / 2.0)
    return 2.0 * calculate_arcsin(reduced_input)
```

Exact excerpt from the final problem5.py implementation.

Taylor-Series Recurrence

```
def calculate_arcsin(x):  
    if x == 0.0:  
        return 0.0  
  
    result = 0.0  
    coefficient = 1.0  
    power = x  
    iteration = 0  
  
    while iteration < MAX_ITERATIONS:  
        term = coefficient * power  
        result = result + term  
  
        if absolute_value(term) <= TOLERANCE:  
            return result  
  
        iteration = iteration + 1  
        coefficient = (  
            coefficient  
            * (2 * iteration - 1)  
            * (2 * iteration - 1)  
            / ((2 * iteration) * (2 * iteration + 1))  
        )  
        power = power * x * x  
  
    raise ConvergenceError(  
        "Taylor-series calculation did not converge."  
    )
```

Newton Square-Root Calculation

```
def calculate_square_root(value):  
    if value < 0.0:  
        raise InputValidationError(  
            "Square root input must be greater than or equal to zero."  
        )  
  
    if value == 0.0:  
        return 0.0  
  
    guess = 1.0  
    iteration = 0  
  
    while iteration < SQRT_MAX_ITERATIONS:  
        next_guess = 0.5 * (guess + value / guess)  
  
        if absolute_value(next_guess - guess) <= SQRT_TOLERANCE:  
            return next_guess  
  
        guess = next_guess  
        iteration = iteration + 1  
  
    raise ConvergenceError(  
        "Square-root calculation did not converge."  
    )
```

Newton's method calculates $\sqrt{(1-x)/2}$ without using `math.sqrt()`.

Verification and Evidence

Accuracy results, repository, requirements, and demonstration

Accuracy Requirement Met

All tested valid inputs satisfy:

$$|\text{program result} - \text{reference value}| \leq 10^{-6} \text{ radians}$$

Input x	Program result	Reference value	Absolute error
−1.0000	3.141592653590	3.141592653590	0.00e+00
−0.9999	3.127450400112	3.127450400112	0.00e+00
0.0000	1.570796326619	1.570796326795	1.76e-10
0.5000	1.047197551172	1.047197551197	2.48e-11
0.9990	0.044725087169	0.044725087169	8.33e-17
0.9999	0.014142253478	0.014142253478	7.81e-17
1.0000	0.000000000000	0.000000000000	0.00e+00

Reference values were generated using Python `math.acos()` only for verification; the application does not call this function.

Public GitHub Repository

github.com/md-aliyaware-khan/SOEN-6011-D2-arccos-calculator

Contents at Submission

- README.md
- problem5.py
- Documentation

Live Demonstration

- Valid and endpoint input
- Invalid-input recovery
- Another calculation without restart
- Clear and Exit controls

Documentation, source code, and meaningful commits are publicly available through the GitHub repository.

Demonstration Checklist

Case	Input	Expected Outcome
Normal calculation	0.5	Displays approximately 1.0471975512 radians.
Near upper endpoint	0.9999	Displays approximately 0.0141422535 radians.
Lower endpoint	-1	Displays π radians.
Outside domain	2	Displays a domain-validation error.
Non-finite input	nan	Displays a finite-number error.
Invalid text	hello	Displays a numeric-input error.
Recovery	Error, then 0	Accepts new input without restarting.
Exit	Exit button	Closes the application.

Selected Requirement-to-Code Traceability

Requirement	Final Implementation Evidence
FR-001-003	ArccosCalculatorGUI, entry field, Calculate button, and Enter-key binding.
FR-004-005	<code>is_finite_number(x)</code> and domain checks in <code>calculate_arccos(x)</code> .
FR-006-008	<code>calculate_arccos()</code> , Taylor-series <code>calculate_arcsin()</code> , and radians output label.
FR-009, FR-011	GUI catches errors, displays feedback, and accepts another calculation.
FR-010	Direct returns for $x = 1$ and $x = -1$.
FR-012-013	<code>clear_fields()</code> and <code>self.window.destroy()</code> .
NFR-002	Numerical-verification table with tested error below 10^{-6} .
NFR-003-004	Iteration limits plus separate GUI, validation, and numerical functions.

Scope of This Table

Selected examples are shown; the repository contains the complete final implementation.

Complete Functional Requirements I

ID	Requirement
FR-001	The system shall provide a graphical user interface implemented using Tkinter.
FR-002	The graphical user interface shall provide a text-entry field through which the user can enter one real-number value x .
FR-003	The system shall provide a Calculate button that initiates calculation of $\arccos(x)$.
FR-004	The system shall validate that the input is a finite numeric value.
FR-005	The system shall validate that the input value is within the domain $[-1, 1]$, inclusive.
FR-006	For valid input, the system shall calculate $\arccos(x)$ using a numerical implementation written from scratch in Python.
FR-007	The system shall not use built-in or library functions for the mathematical computation of $\arccos(x)$ or its subordinate numerical functions.

Complete Functional Requirements II

ID	Requirement
FR-008	The system shall display the calculated principal result in radians, where $0 \leq \arccos(x) \leq \pi$.
FR-009	If a nonnumeric, non-finite, or out-of-domain value is entered, the system shall display a helpful error message explaining the problem and valid input condition.
FR-010	The system shall handle $x = 1$ and $x = -1$ directly, returning 0 and π , respectively.
FR-011	The system shall allow the user to correct invalid input and perform another calculation without restarting the application.
FR-012	The system shall provide a Clear control that removes current input and result or error feedback.
FR-013	The system shall provide an Exit control that closes the application.

Complete Non-Functional Requirements

ID	Requirement
NFR-001	The application shall execute using Python 3.x in an environment that supports Tkinter, without requiring a particular IDE.
NFR-002	For tested valid inputs, the absolute error compared with a trusted reference value shall not exceed 10^{-6} radians.
NFR-003	The Taylor-series and square-root calculations shall stop and display a convergence-related error message if configured iteration limits are reached.
NFR-004	The source code shall separate GUI event handling, validation, numerical calculation, and result or error display into distinct functions.
NFR-005	Error messages shall identify the cause of the problem and state the valid input condition where applicable.
NFR-006	The GUI shall provide readable input, Calculate, Clear, Exit, result, and error-feedback areas.

GAI and CASTROFF: Problems 5–7

Problem 5 – Implementation Review

Prompt: Review a from-scratch Taylor-series $\arccos(x)$ Tkinter implementation; identify endpoint-convergence risks, validation gaps, and tests for a 10^{-6} -radian target.

Decision: Accepted range reduction; added custom exceptions and iteration limits; rejected built-in inverse-trigonometric functions.

Problem 6 – Repository and README

Prompt: Draft a concise README for a Python Tkinter $\arccos(x)$ calculator: prerequisites, run command, method, testing, and limitations.

Decision: Accepted README structure; corrected the command to `python problem5.py`; removed nonexistent-file references.

Problem 7 – Requirements Revision

Prompt: Review D2 requirements for clarity, testability, and consistency with Tkinter and the from-scratch numerical method.

Decision: Revised subjective statements into testable requirements and linked them to final implementation evidence.

CASTROFF applied: context, task, constraints, expected output, validation criteria, and review against final evidence.

Delivered

- Tkinter GUI replacing the D1 text interface
- From-scratch numerical $\arccos(x)$
- Validation, recovery, Clear, and Exit controls

Verified

- Range reduction improves endpoint convergence
- All tested values met 10^{-6} -radian accuracy
- Public repository includes source and README

F1: $\arccos(x)$ delivered as a verified Tkinter application.

References I

-  Concordia University, Department of Computer Science and Software Engineering, *SOEN 6011 Project Description*, Summer 2026.
-  ISO/IEC/IEEE, *ISO/IEC/IEEE 29148: Systems and Software Engineering – Life Cycle Processes – Requirements Engineering*, 2018.
-  James Stewart, *Calculus: Early Transcendentals*, Cengage Learning.
-  Richard L. Burden and J. Douglas Faires, *Numerical Analysis*, Cengage Learning.
-  Python Software Foundation, *Python Documentation*, available at: <https://docs.python.org/3/>.
-  Python Software Foundation, *Tkinter — Python Interface to Tcl/Tk*, available at: <https://docs.python.org/3/library/tkinter.html>.
-  GitHub, *GitHub Documentation*, available at: <https://docs.github.com/>.